

AI ROBOTICS LAB

Full Technical Documentation

Architecture • AI • Robotics • Safety • Research • Claude Agents • Engineering • Data • Deployment • Operations

Master implementation and research reference

This PDF consolidates the project documentation into one navigable technical reference. Open research questions remain explicitly identified rather than being silently resolved.

00 PROJECT

01_Project_Vision

Project Vision and Thesis

The project investigates resource-efficient embodied intelligence: a modular robot brain combining a physics-informed World Model, fast System 1, slower System 2, Awareness Harness, memory/skills, and continual learning. The long-term direction is an embodiment-agnostic robot brain. The architecture is a research hypothesis, not an assumed result.

02_Scope_and_Boundaries

Scope and Boundaries

In scope: simulation-first robotics intelligence, world models, fast/slow control, awareness, memory, skills, continual learning, transfer, resource efficiency, deterministic safety, and agent-assisted implementation. Out of scope for the initial system: claims of consciousness, unrestricted self-modification, learned bypass of safety, and premature universal embeddings.

03_Glossary

Glossary

System 1 = fast learned controller. System 2 = slower planner/reasoner. Awareness Harness = meta-level monitoring and escalation mechanism. World Model = predictive model of relevant dynamics. Surprise = prediction/reality discrepancy. Skill = reusable procedure/policy. Embodiment Adapter = body-specific interface. Safety Kernel = deterministic final actuator gate. Continual Learning = learning from ongoing experience while managing forgetting.

04_Documentation_Map

Documentation Map

Project -> Requirements -> Architecture -> AI -> Robotics -> Safety -> Research -> Agents -> Engineering -> Data -> Experiments -> Deployment -> Operations -> Project Management. This hierarchy moves from why the system exists to what it must do, how it is implemented, how it is tested, and how it is operated.

01 REQUIREMENTS

01_System_Requirements

System Requirements

The system shall represent state, predict consequences, produce fast actions, support slow planning, monitor confidence/calibration/novelty/surprise/stakes/budget, apply deterministic safety, record outcomes, learn from experience, support simulation before hardware, and expose research telemetry. It shall be modular, testable, reproducible, observable, configurable, portable, maintainable, and capable of offline real-time operation.

02_Functional_Requirements

Functional Requirements

REQ-STATE: state and belief representation. REQ-WM: prediction and uncertainty. REQ-S1: low-latency control and confidence. REQ-S2: planning and skill selection. REQ-AH: monitoring and arbitration. REQ-SAFE: limits, watchdogs, emergency handling. REQ-MEM: episodic memory and skills. REQ-CL: online learning and forgetting control. REQ-SIM: deterministic simulation. REQ-ROS: robot integration. REQ-LOG: telemetry and experiment records.

03_Acceptance_Gates

Acceptance Gates

Gate 0 repository/contracts; Gate 1 simulation; Gate 2 World Model; Gate 3 System 1; Gate 4 Awareness; Gate 5 System 2; Gate 6 learning; Gate 7 hardware; Gate 8 research evaluation. Each gate requires implementation, tests, observability, documentation, and reproducibility evidence.

04_Traceability

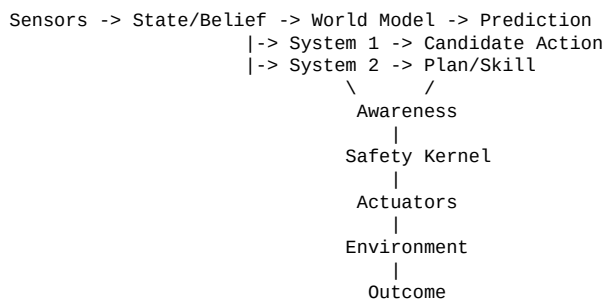
Requirements Traceability

Every implementation task maps to requirement IDs, a component, tests, and acceptance evidence. Contract changes require compatibility review. Safety changes require Safety review. Research mechanisms require hypotheses and metrics.

02 ARCHITECTURE

01_System_Architecture

System Architecture



System 1 is the fast controller; System 2 is slower deliberation; Awareness monitors trust conditions; Safety has final authority.

02_Control_Authority

Control Authority and Arbitration

System 2 proposes goals/plans. System 1 or skills propose actions. Awareness can accept, escalate, request prediction/replanning, or abstain. Safety is final authority. Safety rejection is final for that cycle. Timeouts enter a configured safe state. Learned modules cannot disable hard safety constraints.

03_State_and_Data_Model

State and Data Model

State layers: physical state, belief state, task state, meta state. Canonical records include observation, prediction, uncertainty, action, confidence, plan, surprise, novelty, stakes, safety decision, final action, and outcome. Measurement, estimation, model, policy, and outcome uncertainty must remain distinguishable. Schemas are versioned.

04_Communication_Protocol

Communication Protocol

Message classes: Observation, StateUpdate, PredictionRequest/Result, ActionProposal, PlanProposal, AwarenessDecision, SafetyDecision, Outcome, LearningEvent, MemoryQuery/Result, ExperimentEvent. Required metadata: message ID, timestamp, source, destination, schema version, cycle ID, correlation ID, payload. No undocumented contracts.

05_Runtime_and_Failure_Architecture

Runtime and Failure Architecture

Runtime modes: offline, simulation, HIL, supervised robot, autonomous robot. Critical failures include sensor, estimation, world model, S1, S2, awareness, memory, communication, actuator, and safety failures. Each needs detection, safe response, logging, and recovery/shutdown policy.

06_ADRs

Architecture Decision Records

ADR-001 modular architecture for ablation/replacement. ADR-002 fast/slow control for efficient deliberation. ADR-003 deterministic Safety Kernel outside learned authority. ADR-004 World Model as first-class predictive subsystem. ADR-005 shared representation treated as experimental rather than assumed universal.

03 AI

01_AI_Architecture

AI Architecture

The AI stack is State/Perception -> World Model + System 1 + System 2 -> Awareness -> Memory/Skills -> Continual Learning. The stack is designed around experiential action-consequence learning rather than only train-then-deploy.

02_World_Model

World Model

Initial research direction: object-centric latent physical variables, a hand-written integrator, learned residuals for friction/contact, and progressive comparison with more learned physics models. Evaluate prediction accuracy, uncertainty, compute cost, and usefulness to control and awareness.

03_System_1

System 1

Low-latency learned controller with explicit action space, confidence estimation, calibration, bounded latency, versioned policy, and safe fallback. Evaluate success, latency, calibration, robustness, and recovery after surprise.

04_System_2

System 2

Slow goal interpretation, task decomposition, planning, reasoning, and skill selection. It does not directly command actuators. Evaluate plan quality, invocation efficiency, compute, latency, and contribution over S1 alone.

05_Awareness_Harness

Awareness Harness

Inputs: confidence, calibration, surprise, novelty, stakes, budget, context. Outputs: accept fast action, request System 2, request prediction, request replanning, abstain, or escalate. Version 1 should use interpretable rules; later learned arbitration is a research question. Awareness here means self-monitoring, not consciousness.

06_Memory_Skills

Memory and Skills

Memory includes episodic experiences, retrieval and replay. Skills are reusable and potentially hierarchical procedures. Retrieval provenance and skill versions must be observable. Skill changes require evaluation.

07_Continual_Learning

Continual Learning

Experience -> selection -> replay -> update -> validation -> deployment. Control catastrophic forgetting, unstable updates, distribution shift, and unsafe behaviour using held-out regression, replay, versioning, rollback, and safety gates.

08_Representation_and_Embodiment_Transfer

Representation and Embodiment Transfer

A shared representation is a research direction. It should be physically/task grounded and evaluated for transfer, compositionality, robustness and cost. Embodiment adapters translate shared representations to body-specific actions. Transfer must be distinguished from retraining.

04 ROBOTICS

01_Robotics_Architecture

Robotics Architecture

Shared intelligence connects through an embodiment adapter to robot interfaces, sensors and actuators. Body-specific code stays below shared intelligence interfaces.

02_Environment_and_Simulation

Environment and Simulation

Environment defines observations, actions, state, dynamics, objects, disturbances, task objectives, uncertainty, reset and termination. Simulation should support deterministic seeds, configurable physics, sensor/actuator models, disturbances, domain randomization and reproducible logging. MuJoCo/MJX or equivalent is the initial direction.

03_Sensors_Actuators

Sensors and Actuators

Sensors expose timestamp, measurement, frame, uncertainty, health and calibration metadata. Actuators expose commands, limits, status and fault state. Every actuator command passes the Safety Kernel.

04_ROS2_and_Embodiment_Adapter

ROS2 and Embodiment Adapter

ROS2 can provide integration for nodes, topics/services/actions, lifecycle, timing, drivers and simulation bridges. Core research logic should remain testable without ROS2. The embodiment adapter isolates body-specific action mappings.

05 SAFETY

01_Safety_Architecture

Safety Architecture

Learned intelligence produces candidate actions; the deterministic Safety Kernel validates them before actuation. Safety is outside learned authority.

02_Safety_Requirements

Safety Requirements

Enforce actuator/workspace limits, stale-command detection, communication checks, emergency stop, malformed-command rejection, fault telemetry, safe timeout behaviour, and protection against learned disabling of hard limits.

03_Watchdogs_EStop_Recovery

Watchdogs, Emergency Stop and Recovery

Watch control heartbeat, actuator communication, state freshness and safety health. Emergency stop must be independent of learned decision quality. Recovery may retry, reinitialize, request replanning, switch skill, degrade, or shut down only where policy permits.

04_Fault_Injection

Fault Injection

Test sensor dropout, stale state, malformed messages, actuator disconnect, S1/S2/World Model/Awareness failure, and communication timeout. Each test requires expected safe behaviour and evidence.

06 RESEARCH

01_Research_Program

Research Program

Central question: can modular embodied intelligence learn from experience and allocate computation adaptively under resource constraints? Themes include experiential intelligence, predictive world models, fast/slow control, awareness, continual learning, skills, transfer, and compute efficiency.

02_Research_Questions

Research Questions

Which frontier abstractions transfer into constrained robotics? How does continual learning compare with train-then-deploy? What minimum compute is useful? Can S1/S2 cooperate efficiently? Can prediction error identify untrustworthy situations? Can skills transfer? What representation supports shared intelligence without losing grounding?

03_Hypotheses

Hypotheses

H1 S1+S2 broadens task capability at acceptable compute. H2 Awareness reduces unnecessary S2 use while improving difficult cases. H3 prediction error helps detect states requiring reconsideration. H4 continual learning improves future performance while controlling forgetting. H5 shared representation plus adapters enables measurable cross-embodiment transfer.

04_Baselines_Ablations

Baselines and Ablations

Baselines: task-specific/conventional where applicable, S1 only, S1+S2, S1+World Model, S1+S2+Awareness, full system. Ablate Awareness, World Model, S2, Memory, continual learning, novelty, surprise, stakes and budget inputs.

05_Metrics_Evaluation

Metrics and Evaluation

Performance: success, time, failure, recovery. Decision: S2 invocation, correct/unnecessary escalation, abstention. Prediction: error and calibration. Resources: latency, memory, compute, energy where measurable. Learning: forgetting and transfer. Every result includes baseline, sample count, variability, configuration and failure analysis.

06_Reproducibility_and_Failure_Analysis

Reproducibility and Failure Analysis

Record Git commit, model/config versions, seed, environment, dataset, dependencies, hardware, experiment ID and artifacts. Failure reports contain cause hypothesis, evidence, reproduction, impact, mitigation and effect on the research hypothesis.

07_Experiment_Runbook

Experiment Runbook

Define hypothesis -> select baseline -> freeze versions -> set seed -> validate environment -> pilot -> run trials -> collect telemetry -> analyze -> failure analysis -> archive -> conclude. Evaluation definitions must not be changed after results without recording the change.

08_Experiment_Catalog

Experiment Catalog

E1 S1 baseline. E2 S1+S2. E3 awareness-guided escalation. E4 World Model utility. E5 continual learning. E6 embodiment transfer. Each experiment compares against fixed baselines and reports performance plus compute/latency/complexity costs.

07 AGENTS

01_Agent_Architecture

Claude Agent Architecture

Thirteen roles: Project Lead, Research Governor, Contracts, Simulation, World Model, System 1, Awareness, System 2, Memory & Skills, Safety, Continual Learning, Testing & Evaluation, Integration & DevOps. Agents implement the architecture; they do not silently redesign it.

02_Agent_Operating_Manual

Agent Operating Manual

Workflow: inspect repository -> read contracts -> implement minimal change -> tests -> validation -> document decisions -> completion report -> handoff. Forbidden: silent architecture redesign, undocumented contract drift, skipped tests, unsupported completion claims, safety-limit changes to pass tests, or metric manipulation.

03_Task_and_Communication_Protocol

Agent Task and Communication Protocol

Task fields: task ID, agent, objective, inputs, dependencies, constraints, allowed/forbidden files, acceptance criteria, tests, deliverables. Completion: status, changed files, tests, architecture changes, decisions, limitations, risks, dependencies, next task.

04_Dependency_and_Review

Agent Dependency and Review

Contracts precede dependent implementations. Simulation, World Model and S1 can proceed once interfaces stabilize. Awareness depends on S1/WM contracts. System 2, memory and continual learning depend on stable core interfaces. Safety and Testing cross-cut all phases. Research changes require Research Governor review; safety changes require Safety review; contract changes require Contracts review.

05_Agent_Prompts

Agent Prompt Roles

Project Lead: coordinate. Research Governor: protect scientific validity. Contracts: own interfaces. Simulation: environments. World Model: prediction. S1: fast control. Awareness: arbitration. S2: planning. Memory/Skills: experience and reusable procedures. Safety: hard safety. Continual Learning: updates/forgetting. Testing: verification. DevOps: reproducibility/deployment.

08 ENGINEERING

01_Repository_and_Development

Repository and Development Guide

Recommended repository: src/contracts, state, world_model, system1, system2, awareness, memory, skills, learning, safety, simulation, robot; tests; experiments; configs; docs; scripts; docker; .claude. Development: contract -> tests -> implementation -> integration -> ADR if needed -> report.

02_APIs_Configuration_Logging

APIs, Configuration and Logging

Public APIs define inputs, outputs, errors, timing, version and side effects. Configuration defines mode, robot/environment, model versions, frequencies, thresholds, safety limits, logging and experiment ID. Logs contain timestamp, component, level, cycle/event ID, model version, decision, reason and latency.

03_Testing_CI_CD

Testing and CI/CD

Testing levels: unit, contract, component, integration, simulation, fault injection, regression, research benchmark. CI runs lint/format, unit and contract tests, integration where practical, schema validation, reproducibility checks and container builds. Hardware deployment is gated.

04_Docker_and_Dependencies

Docker and Dependencies

Pin dependencies and provide reproducible development, simulation, experiment and CI environments. Keep hardware-specific dependencies separated where practical. Record direct versions.

09 DATA

01_Data_Architecture

Data Architecture

Observation -> State -> Action -> Outcome -> Experience -> Memory/Replay -> Learning -> Evaluation. Raw and derived data are distinguishable and versioned.

02_Experience_and_Memory_Format

Experience and Memory Format

Experience contains episode ID, state, action, prediction, prediction error, outcome, task result, confidence, novelty, stakes and safety decision. Memory adds context, skill association, timestamp, reliability and provenance. Experiences are immutable after collection.

03_Data_Versioning_and_Quality

Data Versioning and Quality

Record source, schema, version, collection conditions, preprocessing and splits. Validate completeness, timestamps, schema, sensor health, state/action consistency, corruption and duplicates. Do not silently discard low-quality data.

10 DEPLOYMENT OPERATIONS

01_Deployment

Deployment

Stages: simulation -> HIL -> bench hardware -> constrained motion -> supervised low-risk tasks -> expanded operation. Each stage requires acceptance evidence, safety checks, reproducible configuration and rollback.

02_Operations_Runbook

Operations Runbook

Before: configuration, robot/environment, safety, logging, versions. During: loop, health, resources, safety, prediction errors. After: save logs, archive experience, record incidents, verify shutdown.

03_Incident_Response

Incident Response

Stop/safe state -> preserve logs -> incident ID -> classify -> reproduce if safe -> root-cause hypothesis -> mitigation -> regression test -> lessons learned.

04_Troubleshooting_and_Maintenance

Troubleshooting and Maintenance

Start with configuration, contracts, timestamps, component health, logs, versions and environment. Never change safety limits as a troubleshooting shortcut. Maintain dependencies, schemas, models, environments, drivers, documentation and experiment records.

11 PROJECT MANAGEMENT

01_Roadmap_and_Milestones

Roadmap and Milestones

Phase 1 foundation. Phase 2 online continual learning. Phase 3 multi-task/shared models. Phase 4 scaling and documentation. Milestones: contracts, simulator, World Model, S1, Awareness, S2, memory/skills, continual learning, benchmark, hardware, transfer study.

02_Work_Breakdown

Work Breakdown Structure

Governance; contracts; simulation; World Model; S1; Awareness; S2; memory/skills; continual learning; safety; testing; integration; research; hardware; documentation.

03_Risk_Register

Risk Register

Major risks: over-complex architecture, underspecified awareness, inaccurate World Model, catastrophic forgetting, excessive S2 cost, poor embedding geometry, hardware failure, agent contract drift, and implementation overtaking research. Mitigations include modularity, explicit experiments, safety kernel, replay/regression, budget controls, semantic contracts, and research governance.

04_Decision_and_Research_Log

Decision and Research Log

Decision log records ID, issue, alternatives, decision, rationale, consequences and owner. Research log records question, hypothesis, setup, result, unexpected behaviour, interpretation and next experiment. Negative results are preserved.